РУКОВОДСТВО ПОЛЬЗОВАТЕЛЯ · USER GUIDE

# UNEARTH

Smart LFG-to-Energy & Computational Grid Simulator — симулятор добычи свалочного газа, генерации электроэнергии и питания модульных дата-центров для Бишкекского полигона. Что делает каждый модуль, как запустить систему и работать с дашбордом.

---

Версия MVP 0.1 · Backend: Python · FastAPI · Frontend: React · TypeScript · Октябрь 2026

# Содержание

**1. О системе и этом руководстве** **3**

**2. Что нужно для запуска** **3**

2.1. Запуск backend (FastAPI + ML) . . . 4

2.2. Запуск frontend (React + Vite) . . . 4

2.3. Проверка корректности установки . . . 4

**3. Архитектура и структура проекта** **4**

**4. Backend: что делает каждая функция** **5**

4.1. landgem.py — физическое ядро LandGEM (EPA) . . . 5

4.2. seasonal.py — гибридный ML-прогноз с сезонностью . . . 6

4.3. market.py — синтетический рынок и ML-прогноз доходности . . . 6

4.4. optimizer.py — Smart Grid балансировщик . . . 7

4.5. maintenance.py — износ ГПУ и скрубберы . . . 7

4.6. finance.py — финансы и Green Tracker . . . 8

4.7. sectors.py — карта секторов полигона . . . 8

**5. REST API** **9**

5.1. Основные параметры запросов . . . 9

5.2. Пример вызова (PowerShell) . . . 10

**6. Дашборд: как пользоваться** **10**

6.1. ML Controls (боковая панель) . . . 10

6.2. Карточки показателей (Top Cards) . . . 11

6.3. Графики . . . 11

6.4. Распределение мощности (Smart Grid) . . . 11

6.5. Карта секторов и обслуживание . . . 11

**7. Типовые сценарии** **12**

7.1. Первый прогон (дефолт Бишкека) . . . 12

7.2. «А если BTC упадёт?» . . . 12

7.3. Осенний пик влажности и $H_2S$ . . . 12

7.4. Ручное распределение . . . 12

7.5. Окупаемость для инвестора . . . 12

**8. FAQ и устранение неполадок** **13**

**9. Константы и допущения** **13**

# 1. О системе и этом руководстве

UNEARTH — Smart LFG-to-Energy & Computational Grid Simulator — аналитическая B2B/G2G платформа для Бишкекского санитарного полигона. Система моделирует добычу свалочного газа (метана) из «старого тела» свалки, его конверсию в электроэнергию автономными газопоршневыми установками (Jenbacher) и питание модульных дата-центров (ИИ-серверы и ASIC-майнеры) прямо на полигоне, без подключения к сети города.

Цепочка модели: **старый мусор → подземный метан → электроэнергия → вычисления (ИИ / майнинг)**; побочный поток — экология: предотвращённые выбросы $CO_2$-эквивалента и «Фонд озеленения» (5 % чистой прибыли на саженцы).

Руководство адресовано всем, кто работает с системой: аналитику, который задаёт параметры полигона; инженеру, который читает прогноз и обслуживание; инвестору, которому важны окупаемость и риски. Разделы 2–3 описывают установку и архитектуру, раздел 4 — что делает каждая функция backend-ядра, раздел 5 — REST API, раздел 6 — работу с дашбордом, раздел 7 — типовые сценарии.

> **Ключевые исходные данные (проектное досье UNEARTH, ЕБРР / Мэрия / EPA)**
> 20 000 000 т накопленных ТБО · приток 350–400 тыс. т/год · $L_0$ = 70–80 $м^3$/т · k = 0.02–0.04 1/год (аридный климат Чуйской долины) · доля $CH_4$ в газе 50–55 % · 1 $м^3$ $CH_4$ ≈ 3.5 кВт·ч электроэнергии · Antminer S19 Pro — 3.25 кВт · ИИ-сервер — 1.5 кВт · амортизация оборудования 1.5–2 цента за кВт·ч.

Состоит система из трёх модулей: (1) математическое ядро прогнозирования метана (LandGEM + ML-сезонность), (2) экономический калькулятор и умная балансировка нагрузки (Smart Grid, предиктивное обслуживание, финансы), (3) визуальный дашборд на React. Симуляция запускается с дашборда одной кнопкой и проходит по цепочке: прогноз → распределение мощности → обслуживание → финансы → карта секторов.

# 2. Что нужно для запуска

| Компонент | Требование | Примечание |
|---|---|---|
| **Python** | 3.11 или новее | Проверено на 3.13 (Windows). Зависимости: FastAPI, Pandas, scikit-learn, NumPy. |
| **Node.js** | 18 или новее | Проверено на Node 24. Нужен только для фронтенда. |
| **Браузер** | Chrome / Edge / Firefox | Современная версия с поддержкой ES2022. |
| **Порты** | 8000 и 5173 свободны | 8000 — backend (FastAPI), 5173 — dev-сервер Vite. |
| **Диски/сеть** | около 500 МБ | pip-пакеты (NumPy, Pandas, scikit-learn) и npm-пакеты. |

Таблица 1. Минимальные требования к окружению

### 2.1. Запуск backend (FastAPI + ML)

```
cd D:\UNEARTH\backend
python -m pip install -r requirements.txt
python -m uvicorn app.main:app --reload --port 8000
```

Команды запускаются из папки backend; --reload можно опустить для постоянной работы.

После старта откройте Swagger-документацию: **http://localhost:8000/docs** — там же можно вызывать эндпоинты вручную. Проверка живости: **http://localhost:8000/api/health** должен вернуть JSON со статусом «ok».

### 2.2. Запуск frontend (React + Vite)

```
cd D:\UNEARTH\frontend
npm install
npm run dev
```

Dev-сервер поднимется на http://localhost:5173 и проксирует /api на localhost:8000.

Откройте **http://localhost:5173**: дашборд сам запустит первую симуляцию с дефолтными параметрами Бишкека. Кнопка «Запустить симуляцию» повторяет прогон с текущими значениями боковой панели. Продакшен-сборка: **npm run build** (strict TypeScript + Vite), результат — в frontend/dist.

### 2.3. Проверка корректности установки

```
# unit-тесты backend (28 тестов: физика, прогноз, балансировщик, API)
cd D:\UNEARTH\backend
python -m pytest tests -q

# сборка frontend с проверкой типов
cd D:\UNEARTH\frontend
npm run build
```

Оба шага в репозитории уже выполнены и проходят: **28 passed** у pytest и успешная сборка Vite без ошибок типов.

## 3. Архитектура и структура проекта

| Путь | Назначение |
|---|---|
| **backend/app/main.py** | Точка входа FastAPI: CORS, маршруты, описание сервиса. |
| **backend/app/config.py** | Единый источник констант: физика из досье + экономические допущения. |
| **backend/app/schemas.py** | Pydantic-модели запросов (контракты API). |

| Путь | Назначение |
|---|---|
| **backend/app/api/routes.py** | REST-эндпоинты и сквозной маршрут /api/simulation/run. |
| **backend/app/engines/** | Движки расчётов: landgem, seasonal, market, optimizer, maintenance, finance, sectors. |
| **backend/tests/** | 28 pytest-тестов: физика, прогноз, балансировщик, обслуживание, API. |
| **frontend/src/store/** | Zustand-стор: оркестрация асинхронных ML-запросов. |
| **frontend/src/components/** | UI: дашборд, боковая панель, карточки, графики, карта секторов. |
| **frontend/src/api/client.ts** | HTTP-клиент к REST API (Vite-прокси в dev, VITE_API_URL в прод). |

Таблица 2. Карта репозитория

Данные двигаются так: дашборд отправляет параметры полигона в **/api/forecast** — тот считает физическую кривую LandGEM и накладывает ML-сезонность. Средняя мощность первого года уходит в **/api/optimize** (балансировщик), **/api/maintenance** (износ и OPEX) и **/api/sectors** (карта) — эти три запроса выполняются параллельно. Из результатов балансировщика и обслуживания складывается **/api/finance** (Монте-Карло, окупаемость, Green Tracker). В Zustand-сторе это три последовательных раунда запросов с раздельными статусами загрузки и ошибок.

# 4. Backend: что делает каждая функция

## 4.1. landgem.py — физическое ядро LandGEM (EPA)

Модель первого порядка распада: $Q(t) = \Sigma\, k \cdot L_0 \cdot M_i \cdot e^{(-k \cdot age)}$. «Старое тело» — 20 млн т, накопленные равномерно за 54 года (с 1972 г.); ежегодный приток даёт стационарный вклад $\approx L_0 \times (M/12)$ $м^3$/мес, потому что депонирование идёт непрерывно уже десятилетия. Расчёт ведётся по месяцам на 15–20 лет вперёд.

| Функция | Что делает |
|---|---|
| **monthly_ch4_m3(inp)** | Главная кривая: образование $CH_4$ по месяцам горизонта, $м^3$/мес. |
| **monthly_ch4_history(inp, n)** | Тренд за 48 месяцев до старта — обучающая выборка ML-слоя. |
| **monthly_energy_mwh(q, inp)** | Переводит $м^3$ $CH_4$ в МВт·ч: × отбор газа × готовность ГПУ × 3.5 кВт·ч. |
| **avg_power_mw(e, hours)** | Средняя электрическая мощность месяца, МВт (по умолчанию 730 ч). |
| **legacy_cohorts(inp)** | Разбивает 20 млн т на годовые когорты с возрастом для затухающей суммы. |

Таблица 3. Функции физического ядра

> **Что настраивается**
>
> Масса (20 млн т), приток (375 тыс. т/год по умолчанию), $L_0$ (75 $м^3$/т), k (0.03), эффективность сбора газа (75 %), готовность ГПУ (92 %), кПД 3.5 кВт·ч/$м^3$ — все поля редактируются с дашборда и приходят в API как LandGEMInput.

## 4.2. seasonal.py — гибридный ML-прогноз с сезонностью

Поверх физической кривой строится «история наблюдений» (48 месяцев с фиксированным seed): тренд × сезонный множитель × AR(1)-шум — имитация мониторинга скважин. Затем декомпозиция: центральное скользящее среднее MA-12 выделяет тренд, средние по календарным месяцам дают эмпирические сезонные индексы, лог-остатки оценивают автокорреляцию ρ (AR(1)) и σ инноваций. Прогноз = физический тренд × сезонный индекс × AR-продолжение; доверительный интервал 95 % растёт с горизонтом.

| Компонент | Что делает |
|---|---|
| **moisture_index(month)** | Влажность массива: осенний пик (арбузы/дыни) +18 % в сентябре–октябре, летняя засуха −4 % в июле. |
| **_synthetic_history(...)** | Генерирует обучающую «историю наблюдений»; осенью волатильность ×2.3. |
| **decompose(obs, months)** | MA-12 декомпозиция, сезонные индексы, оценка AR(1) по лог-остаткам. |
| **forecast_builtin(...)** | Прогноз и 95 % CI встроенной модели (работает без внешних зависимостей). |
| **forecast_prophet(...)** | Альтернатива на Prophet: годовая сезонность + мультипликативный регрессор физтренда. |
| **run_hybrid_forecast(...)** | Полный конвейер; если Prophet недоступен — автоматический откат на builtin с пометкой в meta. |

Таблица 4. Функции слоя прогнозирования

Prophet — опциональный пакет (тяжёлый cmdstan-тулчейн). Встроенная модель полностью реализована и является рабочим путём по умолчанию; ветка Prophet включается флагом use_prophet, когда пакет установлен.

## 4.3. market.py — синтетический рынок и ML-прогноз доходности

Рыночных лент в MVP нет (по ТЗ — синтетические данные): BTC моделируется ансамблем путей GBM с редкими крэшами, аренда GPU — процессом Орнштейна–Уленбека вокруг 2 $/GPU·ч. На этих путях обучаются RandomForestRegressor и HistGradientBoostingRegressor предсказывать доходность BTC на 3 месяца по фичам моментума и волатильности.

| Функция | Что делает |
|---|---|
| **get_market_model(seed)** | Обучает и кэширует ансамбль моделей (детерминирован по seed). |

| Функция | Что делает |
|---|---|
| **predict_return_3m(btc, rent)** | Ожидаемая доходность BTC на 3 мес, ограничена ±35 %. |
| **miner_margin_multiplier(ret)** | Риск-дисконт майнинга: 1 + 0.5·E[ret], диапазон 0.25–1.5. |
| **simulate_future_markets(...)** | 400 будущих траекторий BTC и ренты GPU для Монте-Карло финансов. |

Таблица 5. Функции рыночного блока

## 4.4. optimizer.py — Smart Grid балансировщик

Сначала качество газа превращается в derate-коэффициент доступной мощности: стабильность горения ($CH_4 \geq 40$ %) умножается на штраф за $H_2S$ (выше 50 ppm мощность режется). Затем считается маржа на кВт·день по классам узлов; мощность заполняется по убыванию маржи с учётом PUE контейнеров (1.15) и собственных нужд станции (8 %).

| Функция | Что делает |
|---|---|
| **gas_quality(ch4, h2s)** | Derate мощности, индексы стабильности и штрафа, флаг «стабилен». |
| **auto_allocate(...)** | AI Auto-Optimize: жадное заполнение по марже/кВт, лимит рынка аренды GPU (1000 GPU). |
| **manual_allocate(...)** | Manual Allocation: экономика заданного парка + флаг перегруза. |
| **mining_revenue_usd_day** | Доход Antminer: 144 блока × 3.125 BTC / хешрейт сети × 110 TH × курс, пул −1.5 %. |
| **ai_revenue_usd_day** | Доход ИИ-сервера: рента GPU × 24 ч × 2 GPU. |
| **amortization_usd_day** | Амортизация оборудования 1.75 ¢/кВт·ч (из досье: 1.5–2 ¢). |

Таблица 6. Функции балансировщика

> **Лестница отключений**
>
> При падении мощности или качества газа узлы отключаются по возрастанию маржи на кВт·день: первыми уходят майнеры, ядро ИИ сохраняется. Порядок возвращается в поле shutdown_order, а логика решения — в rationale (человекочитаемые объяснения).

## 4.5. maintenance.py — износ ГПУ и скрубберы

Риск №1 досье — агрессивный газ: сероводород сжигает генераторы. Скорость износа = 0.35 %/мес × ($H_2S$ / 25)^1.35 — нелинейная коррозия. Health = 100 − износ × наработка; ниже 15 % — капремонт. Загрузка серы скрубберов = расход LFG × ppm × плотность $H_2S$; ресурс сорбента = ёмкость / загрузка, замены идут в OPEX.

| Функция / поле | Что делает |
|---|---|
| **wear_pct_per_month(h2s)** | Скорость износа, %/мес, по текущему $H_2S$. |
| **run_maintenance(inp)** | Здоровье каждого Jenbacher (с разбросом наработки ±4 мес), RUL, риск отказа за 12 мес. |
| **scrubber{...}** | Ресурс сорбента (мес), замен в год, стоимость замен. |
| **opex_monthly_usd{...}** | OPEX по статьям: скрубберы, расходники, ФОТ, страхование, амортизация, резерв капремонтов. |

Таблица 7. Функции предиктивного обслуживания

## 4.6. finance.py — финансы и Green Tracker

CAPEX собирается по статьям (скважины, ГПУ 750 $/кВт, скрубберы, подстанция, ЦОД-контейнеры, железо вычислений, резерв 12 %). Выручка моделируется Монте-Карло: 400 траекторий BTC и ренты GPU, рост сложности сети −2 %/мес к доходности майнинга. Отчёт даёт P10/P50/P90 по годам, дисконтированный кумулятивный поток (WACC 10 %), окупаемость и IRR.

| Функция / поле | Что делает |
|---|---|
| **estimate_plant_capex(...)** | Статьи CAPEX станции + инжиниринг и резерв; используется и в обслуживании. |
| **run_finance(inp)** | Монте-Карло выручки/прибыли, окупаемость, IRR, $CO_2$, Фонд озеленения. |
| **totals.green_fund_usd** | 5 % чистой прибыли (медиана) → саженцы по 4 $ (trees_planted). |
| **totals.co2_avoided_t** | Уничтоженный $CH_4$ × 0.716 кг/$м^3$ × GWP-28, тонны $CO_2$-экв. |
| **chart[]** | Готовые точки для графика: Y0 (CAPEX) + годы: OPEX, прибыль, кумулятивный поток. |

Таблица 8. Функции финансового блока

## 4.7. sectors.py — карта секторов полигона

48 секторов (сетка 6 × 8): возрастные классы (свежие/средние/старые), поток газа по долям массы (распределение Дирихле), $H_2S$ по классам с осенней поправкой на влажность. Нестабильность секторов предсказывает RandomForestClassifier, обученный на латентных правилах (осенний пик + высокий $H_2S$). Статусы: green — стабильный газ; amber — усиленная фильтрация; red — высокий $H_2S$/нестабильность; depleted — выработан. Функция build_sectors(...) возвращает сетку, статусы и сводку.

# 5. REST API

| Метод и путь | Назначение |
|---|---|
| **GET /api/health** | Статус сервиса, доступность Prophet, версия. |
| **POST /api/forecast** | LandGEM + ML-прогноз: годовой и месячный ряды, 95 % CI, moisture_index. |
| **POST /api/optimize** | Smart Grid: авто или ручное распределение мощности. |
| **POST /api/maintenance** | Износ ГПУ, скрубберы, OPEX. |
| **POST /api/finance** | Монте-Карло, CAPEX/OPEX, окупаемость, $CO_2$, Фонд озеленения. |
| **POST /api/sectors** | Карта секторов со статусами и рисками. |
| **POST /api/simulation/run** | Сквозной прогон всех модулей одним запросом (удобно для интеграций). |

Таблица 9. Эндпоинты (полные модели запросов — backend/app/schemas.py и Swagger /docs)

## 5.1. Основные параметры запросов

| Поле ForecastRequest | По умолчанию | Смысл |
|---|---|---|
| **total_legacy_tonnes** | 20 000 000 | Накопленные ТБО, т. |
| **annual_inflow_tonnes** | 375 000 | Приток отходов, т/год (350–400 тыс. по досье). |
| **lo_m3_per_tonne** | 75 | Потенциал метана $L_0$, $м^3$/т (70–80). |
| **k_per_year** | 0.03 | Скорость разложения k, 1/год (0.02–0.04). |
| **methane_fraction** | 0.52 | Доля $CH_4$ в свалочном газе (50–55 %). |
| **horizon_years** | 15 | Горизонт прогноза, лет (10–20 на дашборде, 3–25 в API). |
| **use_prophet** | false | Пробовать Prophet, если установлен. |

Таблица 10. Ключевые входы прогноза

| Поле OptimizeRequest | По умолчанию | Смысл |
|---|---|---|
| **available_mw** | обязательное поле | Доступная мощность, МВт (берётся из прогноза, год 1). |
| **ch4_fraction / h2s_ppm** | 0.52 / 30 | Качество газа → derate мощности. |
| **btc_price** | 65 000 $ | Текущий курс для маржи майнинга. |
| **gpu_rent_usd_per_gpu_hour** | 2.0 $ | Аренда GPU для дохода ИИ-серверов. |
| **mode** | auto | auto — AI Auto-Optimize, manual — ручной парк. |

| Поле OptimizeRequest | По умолчанию | Смысл |
|---|---|---|
| **manual_miners / manual_ai_servers** | 0 / 0 | Размер парка в ручном режиме. |
| **gpu_demand_cap** | 1000 | Ёмкость рынка аренды GPU (шт). |

Таблица 11. Ключевые входы балансировщика

### 5.2. Пример вызова (PowerShell)

```
$body = @{ available_mw = 6.5; btc_price = 65000; gpu_rent_usd_per_gpu_hour = 2.0;
           mode = 'auto'; h2s_ppm = 45 } | ConvertTo-Json
Invoke-RestMethod -Uri 'http://localhost:8000/api/optimize' -Method Post `
  -ContentType 'application/json' -Body $body
```

Ответ содержит allocation (по классам узлов), used/curtailed МВт, market (прогноз BTC) и rationale.

## 6. Дашборд: как пользоваться

Дашборд — тёмная тема (slate-900, акценты emerald-500). Слева — панель ML Controls, справа — карточки показателей, графики, распределение мощности, карта секторов и обслуживание. Изменение любого параметра не пересчитывает модель само: настройте значения и нажмите «Запустить симуляцию».

### 6.1. ML Controls (боковая панель)

| Контрол | Что делает |
|---|---|
| **Накопленные ТБО / приток** | Масса «старого тела» и ежегодный приток — база кривой LandGEM. |
| **$L_0$ (70–80 м$^3$/т)** | Потенциал метана тонны отходов: выше — больше газа. |
| **k (0.02–0.04)** | Скорость разложения: выше — быстрее выход на пик и быстрее затухание. |
| **Доля $CH_4$ (45–60 %)** | Качество газа: влияет на derate мощности и потоки LFG. |
| **Горизонт (10–20 лет)** | Длина прогноза и финансовой модели. |
| **$H_2S$ (5–150 ppm)** | Сероводород: износ ГПУ, ресурс скрубберов, derate, лестница отключений. |
| **Курс BTC / аренда GPU** | Рыночные условия: маржа майнинга против дохода ИИ-серверов. |
| **Наработка ГПУ (мес)** | Возраст парка для расчёта health score. |
| **Manual / AI Auto** | Режим распределения: ручные слайдеры парка или ML-балансировщик. |
| **Дефолты Бишкека** | Сброс всех параметров к значениям досье (20 Мт · $L_0$ 75 · k 0.03). |

Таблица 12. Контролы боковой панели

## 6.2. Карточки показателей (Top Cards)

| Карточка | Что показывает |
|---|---|
| **Генерация (год 1)** | Средняя мощность первого года, МВт; энергия за год и пиковая мощность. |
| **Серверы в работе** | Сколько ИИ-серверов и ASIC запущено; выделенная и доступная мощность. |
| **Health Score ГПУ** | Здоровье слабейшего Jenbacher и его остаточный ресурс; цвет — статус. |
| **Green Tracker** | Предотвращённые выбросы $CO_2$-экв., Фонд озеленения и число саженцев. |

Таблица 13. Карточки верхней панели

## 6.3. Графики

**Прогноз метана (ComposedChart)**: пунктирная линия — чистый LandGEM без сезонности; зелёная линия — гибридный ML-прогноз; полупрозрачная область — 95 % доверительный интервал. Наведите курсор на точку, чтобы увидеть точные значения в млн $м^3$ $CH_4$/год. Осенние скачки видно на месячном ряде (в API — monthly[] с moisture_index).

**Финансовая эффективность**: столбцы CAPEX (Y0), OPEX и чистой прибыли (медиана P50) по годам; голубая линия — дисконтированный кумулятивный поток. Окупаемость — точка, где линия пересекает ноль; в шапке карточки дублируются CAPEX, срок и IRR. Диапазон неопределённости P10–P90 — в ответе API (annual[].net_profit_p10_usd / p90).

## 6.4. Распределение мощности (Smart Grid)

Панель показывает по каждому классу узлов количество, мощность, выручку и маржу в сутки; справа — логика ML-балансировщика (rationale), режим рынка, прогноз BTC, маржа на кВт·день и лестница отключений. В режиме **Manual Allocation** сдвиньте слайдеры парка — панель посчитает экономику и предупредит о перегрузе красным баннером. В режиме **AI Auto-Optimize** распределение считает backend по марже на кВт·день.

## 6.5. Карта секторов и обслуживание

| Цвет | Критерий | Что делать |
|---|---|---|
| **Зелёный** | Стабильный газ, $H_2S$ в норме | Обычная эксплуатация. |
| **Оранжевый** | $H_2S$ выше 25 ppm или риск нестабильности | Усиленная фильтрация. |
| **Красный** | $H_2S$ выше 60 ppm или риск выше 0.7 | Приоритет очистки; узлы отключаются первыми. |
| **Серый** | Поток газа ниже порога | Сектор выработан — перебросить скважины. |

Таблица 14. Легенда карты секторов

Клик по сектору открывает карточку: поток газа ($м^3$/ч), $H_2S$, возраст массива и ML-прогноз нестабильности. Яркость клетки — поток газа относительно максимума. Ниже карта дополняется панелью обслуживания: здоровье каждого Jenbacher с прогресс-баром, ресурс сорбента, замены скрубберов и OPEX по статьям.

# 7. Типовые сценарии

## 7.1. Первый прогон (дефолт Бишкека)

Запустите backend и фронтенд (раздел 2), откройте дашборд — симуляция пойдёт автоматически. Проверьте: генерация первого года должна быть в диапазоне единиц МВт (зависит от доли притока), зелёная линия прогноза — выше пунктирной базы в осенние месяцы, финансы — окупаемость в первые годы.

## 7.2. «А если BTC упадёт?»

В боковой панели задайте Курс BTC = 40 000 и запустите симуляцию. ML-модель прогноза рынка даст отрицательное ожидание, множитель маржи майнинга снизится, и балансировщик перераспределит мощность в пользу ИИ-серверов (вплоть до лимита рынка аренды GPU). Сравните карточку «Серверы» и маржу на кВт·день до и после.

## 7.3. Осенний пик влажности и $H_2S$

Поднимите $H_2S$ до 90–120 ppm. Наблюдайте: derate мощности в логике балансировщика, падение health score и остаточного ресурса ГПУ, сокращение ресурса скрубберов и рост OPEX, покраснение секторов на карте. Это демонстрация риска №1 досье: агрессивный газ без очистки сжигает генераторы.

## 7.4. Ручное распределение

Переключите режим в Manual и задайте, например, 800 Antminer и 200 ИИ-серверов. Панель покажет экономику парка; если потребление (с PUE 1.15) превысит доступную мощность — появится баннер перегруза. Сравните с решением AI Auto-Optimize при тех же ценах.

## 7.5. Окупаемость для инвестора

В карточке финансов возьмите CAPEX, срок окупаемости и IRR; на графике найдите год перехода кумулятивного потока через ноль. Стресс-тест: поднимите курс BTC и аренду GPU по очереди — P10/P90 в API покажет разброс. Green Tracker внизу карточек покажет $CO_2$-эффект и фонд озеленения (5 % прибыли).

# 8. FAQ и устранение неполадок

| Проблема | Причина и решение |
|---|---|
| **Дашборд пишет «API недоступен» / баннер «модули без ответа»** | Backend не запущен. Поднимите uvicorn на :8000 (раздел 2.1) и повторите «Запустить симуляцию». |
| **Ошибка «порт 8000 занят»** | Запустите на другом порту: uvicorn app.main:app --port 8001, и укажите VITE_API_URL=http://localhost:8001 для фронта. |
| **npm install падает или тянется вечно** | Проверьте сеть/прокси; повторите npm install — он идемпотентен. При ошибке esbuild выполните npm rebuild esbuild. |
| **Prophet не устанавливается** | Это нормально на Windows/Python 3.13: пакет требует cmdstan-тулчейн. Система автоматически работает на встроенной модели (meta.model = builtin-seasonal-ar1). |
| **Результаты «прыгают» при одинаковых параметрах** | Не должны: seed фиксирован. Проверьте, что не меняли поля «Курс BTC»/«Аренда GPU» между прогонами. |
| **Нужны реальные цены рынка** | Замените генератор в backend/app/engines/market.py на свою ленту (интерфейс — btc/rent матрицы) либо передавайте btc_price и gpu_rent в каждом запросе. |

Таблица 15. Частые вопросы

# 9. Константы и допущения

| Параметр | Значение | Источник |
|---|---|---|
| **Накопленные ТБО** | 20 000 000 т | Досье (ЕБРР/Мэрия) |
| **Приток отходов** | 350–400 тыс. т/год | Досье |
| **$L_0$** | 70–80 $м^3$/т | Досье (EPA) |
| **k** | 0.02–0.04 1/год | Досье (EPA, аридный климат) |
| **Доля $CH_4$** | 50–55 % | Досье |
| **Энергия метана** | 3.5 кВт·ч/$м^3$ | Досье (Jenbacher) |
| **Antminer S19 Pro** | 3.25 кВт, 110 TH/s | Досье (мощность); хешрейт — спецификация |
| **ИИ-сервер** | 1.5 кВт, 2 GPU | Досье (мощность); GPU — допущение MVP |
| **Амортизация оборудования** | 1.5–2 ¢/кВт·ч | Досье (Модуль 2) |
| **CAPEX станции** | скважины 1.9 млн $, ГПУ 750 $/кВт, скрубберы 85 тыс. $/ГПУ и др. | Допущение MVP |

| Параметр | Значение | Источник |
|---|---|---|
| **WACC / налоги** | 10 % / не моделируются (pretax) | Допущение MVP |
| **GWP метана** | 28 (AR5, 100 лет) | Допущение MVP |
| **Фонд озеленения** | 5 % чистой прибыли, саженец 4 $ | Досье (5 %); цена — допущение |
| **Рынок** | BTC GBM (σ 55 %/год), GPU OU вокруг 2 $/ч | Синтетика по ТЗ |

Таблица 16. Ключевые константы (полный список с пометками — backend/app/config.py)

Все значения переопределяются: физические — через параметры API и дашборд, экономические — в backend/app/config.py (константы с пометкой ASSUMPTION). Симуляционные данные не являются инвестиционной рекомендацией.